

Assessment of x3d2

An external submission of x3d2 ([GitHub](#), commit 84c750f38af041661852dc35858c879293146db6 [Faster OpenMP Reorder and Accumulation Kernels \(#329\)](#)) was made on the 8th of July 2026 to the SHAREing team at Durham.

This assessment is performed by Emily Wilkinson and Ananya Gangopadhyay of Durham University between the 8th of July 2026 and the 21st of September 2026.

Disclaimers

1. This report is not a commentary on code quality.
2. The pre-assessment is only a preliminary assessment of submission suitability and does not guarantee a full assessment. It will be provided to the submitter indicating if the full assessment will be undertaken or detail reasons for rejection.

Table of contents

- [Assessment of x3d2](#)
 - [Disclaimers](#)
 - [Table of contents](#)
 - [Pre-assessment of x3d2](#)
 - [Benchmark setup](#)
 - [Fetch and build program](#)
 - [Fetch and run benchmark](#)
 - [Reference architecture](#)
 - [Description of working environment](#)
 - [Hardware information](#)
 - [Libraries and modules](#)
 - [Assessment tools](#)
 - [Compiler setup and optimisations](#)
 - [Computational complexity and scaling](#)
 - [Submitted comments copied direct from form](#)
 - [Memory, storage and I/O](#)
 - [Additional comments from submitter](#)
 - [Submitted comments copied direct from form](#)
 - [Pre-assessment outcome](#)
 - [High-level performance assessment of x3d2](#)
 - [Setup Details](#)
 - [Programming model](#)
 - [SLURM configuration](#)
 - [Base input](#)

- Core-level assessment
- Intra-node assessment
- Inter-node assessment
- GPU Assessment
- I/O Assessment
- Summary
- High-level assessment outcome

Pre-assessment of **x3d2**

Benchmark setup

Fetch and build program

The submitter has requested an assessment of x3d2, commit 84c750f38af041661852dc35858c879293146db6. It can be fetched from GitHub as follows:

```
git clone https://github.com/xcompact3d/x3d2/ && cd x3d2
git checkout 84c750f38af041661852dc35858c879293146db6
```

The CPU version of the program can then be built as follows, yielding an executable at **build-omp/bin/xcompact**:

```
cmake -S . -B build-omp -DCMAKE_BUILD_TYPE=Release -DWITH_ADIOS2=ON
cmake --build build-omp -j
```

Fetch and run benchmark

The benchmark is included in the same git repository as the program. The benchmark file selected is Taylor-Green. To run the benchmark:

```
OMP_NUM_THREADS=3 mpirun -np 8 ./build-omp/bin/xcompact
examples/TGV/input.x3d
```

The following output is expected every 100 iterations:

```
Time for this time step (s): 0.645300210
time = 0.100000000000000001 iteration = 100
enstrophy: 0.37524995458003335
div u max mean: 8.8705084944074031E-014 4.9909836307199695E-015
```

Additionally, the files **decomp_2d_setup.log** and **monitoring.csv** will be generated in the IO-enabled version.

Reference architecture

Reference benchmark architecture for the submitted benchmark case:

- CPU: AMD EPYC 7443 24-Core Processor, 1 socket, 24 physical cores, 48 hardware threads
- NUMA: 4 NUMA domains
- GPU: NVIDIA A100.

Description of working environment

Hardware information

Two different clusters will be used for this assessment. The first is the Hamilton system, and the second is Bede, both based at Durham University.

Hamilton will be used to run the CPU-only version of the submitted code.

The hardware details for Hamilton are available [here](#). This information is corroborated by running `cat /proc/cpuinfo` on one of the compute nodes via an interactive session. There are 120 standard compute nodes on the system:

Specification	Per node
Processors	2 × AMD EPYC 7702
Clock speed per CPU	2000 MHz base clock, 3350 MHz boost clock
Sockets	2
Cores / socket	64
NUMA domains / socket	4
L1/L2 Cache size / core	32KB L1, 512KB L2
L3 Cache size / 4-core chiplet	16MB
Chiplets / NUMA domain	4
Simultaneous Multithreading	Disabled
RAM	256GB DDR4
Local storage	400GB SSD
Interconnect	Infiniband HDR 200GB/s high performance interconnect, with a 2.6:1 fat tree topology formed of non-blocking islands of up to 26 nodes

Additionally, there is a high-memory partition with 2 nodes and a single GPU node in its own partition, which will not be used in this assessment.

Libraries and modules

In order to build the CPU version on the assessment system, Hamilton8, we need to load the following modules:

```
module load gcc openmpi
```

This loads GCC 11.2.0 and OpenMPI 4.1.1, along with cmake 3.26.5 being present by default. A minimum version of cmake 3.21 is required by 2decomp.

Assessment tools

The following tools are available on Hamilton and intended for use for the high-level assessment:

1. likwid 5.5.1
2. darshan 3.5.0
3. GNU time (/usr/bin/time)

Compiler setup and optimisations

The submitted code uses the **cmake** build system. The configuration invocation

```
cmake -S . -B build-omp -DCMAKE_BUILD_TYPE=Release -DWITH_ADIOS2=ON
```

configures the build for release, which adds the following compiler optimisation flags for the CPU build:

```
-O3 -ffast-math -funroll-loops -floop-optimize -march=native
```

No compiler or library versions were specified and therefore a set which successfully compiled the program was selected and is described in the [description of the working environments](#).

Computational complexity and scaling

Submitted comments copied direct from form

The main problem-size parameter is `dims_global = Nx, Ny, Nz`. For cubic cases, increasing N from 256 to 512 increases the number of grid cells by 8x.

The core flow-solver work scales approximately with the number of grid cells times the number of timesteps:

```
Work ~ O(Nx * Ny * Nz * n_iters)
```

For FFT-based Poisson solves and global transposes, the cost also includes communication and FFT complexity. For cubic grids this introduces an approximate FFT contribution of:

```
O(N^3 log N)
```

plus MPI communication/transposition costs that depend on the domain decomposition and interconnect.

To produce strong scaling:

- keep `dims_global` fixed
- increase MPI ranks / OpenMP threads / GPUs
- adjust `nproc_dir` so its product matches the MPI rank count

To produce weak scaling:

- increase `dims_global` as the number of ranks/GPUs increases so that the local grid size per rank/GPU remains approximately constant
- for example, if doubling resources in one decomposition direction, double the corresponding global grid dimension and update `nproc_dir` accordingly

For single-node CPU benchmarks on a 24-core AMD EPYC 7443, useful configurations include:

- 24 MPI ranks x 1 thread
- 12 MPI ranks x 2 threads
- 8 MPI ranks x 3 threads
- 4 MPI ranks x 6 threads

For GPU benchmarks, start with:

- 1 MPI rank per GPU

Memory, storage and I/O

A test run of the program indicates that 4.58 GB of RAM was used for a default (256x256x256) sized run with 8 ranks decomposed 1, 2, 4 over 3000 iterations.

The benchmark writes to the console output every 100 iterations [as indicated by the submitter](#). The benchmark also outputs files totalling 0.02 MB for a run with 3000 iterations. These can be disabled by setting

```
n_output = 0
checkpoint_freq = 0
snapshot_freq = 0
keep_checkpoint = F
restart_from_checkpoint = F
```

or enabled for an I/O assessment run with

```
n_output = 10
checkpoint_freq = 100
snapshot_freq = 100
keep_checkpoint = T
```

Additional comments from submitter

Submitted comments copied direct from form

The submitted Taylor-Green vortex benchmark is intended primarily as a performance benchmark for x3d2 rather than a full physics-validation submission. It exercises the main computational patterns of the code: compact finite-difference operators, pressure projection/Poisson solve, MPI communication/domain decomposition, OpenMP CPU execution, CUDA GPU execution, and optional ADIOS2 I/O.

The primary quantity of interest is the timestep loop performance, excluding or at least separating initial setup where possible.

x3d2 prints timing information including averaged time per timestep. This is the preferred metric for solver performance because it focuses on the repeated compute phase rather than one-off setup costs.

For most performance comparisons, please report:

- total wall time for the full run
- averaged time per step
- number of iterations
- backend and parallel configuration

Pre-assessment outcome

The submitted program is suitable to proceed to a high-level assessment.

High-level performance assessment of **x3d2**

The submitter has specified that out of the five performance dimensions, the ones relevant to this assessment and benchmark are:

- ☒ Core-level assessment
- ☒ Intra-node assessment
- ☒ Inter-node assessment
- ☐ GPU/accelerator assessment (Pending pre-assessment)
- ☒ I/O assessment

We provide brief explanations of the high-level metrics in their relevant sections. However, for a more detailed description, please refer to Chapter 5 in the [performance assessment guidebook](#).

Setup Details

As the code was [successfully compiled on Hamilton](#) during the pre-assessment, the system was used for the high-level assessment as well using the setup, compilers and modules described [there](#).

Programming model

The compiled code uses a hybrid MPI and OpenMP model for both intra-node and inter-node runs. The submitter indicated several options for configuring the MPI ranks and threads without any indicated preference. We noted that for MPI-only runs, where the number of MPI ranks matches the number of cores, runs with 32 ranks and over ended in segmentation faults:

```
Caught signal 11 (Segmentation fault: address not mapped to object at
address $(ADDRESS))
```

We opted to assign one MPI rank per NUMA domain. With 8 NUMA domains per node, MPI ranks were assigned for every 16 cores. For example, for runs with 32 cores, 2 ranks were assigned, and for 128 cores, 8 ranks were assigned, with 16 threads per rank for both cases.

SLURM configuration

All analyses were performed on an exclusive node on Hamilton with 64 GB memory regardless of the number of MPI ranks or OpenMP threads used. For runs on 1 node i.e. 128 core, the configuration is set to:

```
#SBATCH --ntasks-per-node=8
#SBATCH --cpus-per-task=16
#SBATCH --mem=64G
#SBATCH --nodes=1
#SBATCH --exclusive
#SBATCH -p multi
#SBATCH --distribution=block:block
```

For intra-node runs, the `--ntasks-per-node` and `--cpus-per-task` values are set to the number of MPI ranks and OpenMP threads respectively. For inter-node runs, the `--nodes` value is varied.

MPI ranks are distributed in a `block:block` configuration. The following directives were used for OpenMP thread distribution:

```
export OMP_NUM_THREADS=$SLURM_CPUS_PER_TASK # Set with --cpus-per-task
export OMP_PLACES=cores
export OMP_PROC_BIND=close
```

Base input

The following is the input configuration used for serial runs:

```

&domain_settings
! Flow case
flow_case_name = 'tgv'

! Global number of cells in each direction
L_global = 6.283185307179586d0, 6.283185307179586d0, 6.283185307179586d0

! Global domain dimensions
dims_global = 256, 256, 256

! Domain decomposition in each direction
nproc_dir = 1, 1, 1

! BC options are 'periodic' | 'neumann' | 'dirichlet'
BC_x = 'periodic', 'periodic'
BC_y = 'periodic', 'periodic'
BC_z = 'periodic', 'periodic'
/End

&solver_params
Re = 1600d0
time_intg = 'AB3' ! 'AB[1-4]' | 'RK[1-4]'
dt = 0.001d0
n_iters = 1000
n_output = 100
poisson_solver_type = 'FFT' ! 'FFT' | 'CG'
der1st_scheme = 'compact6'
der2nd_scheme = 'compact6' ! 'compact6' | 'compact6-hyperviscous'
interpl_scheme = 'classic' ! 'classic' | 'optimised' | 'aggressive'
stagder_scheme = 'compact6'
/End

```

For parallel runs, the `nproc_dir` parameter was set to match the number of MPI ranks used. For consistency, the default 1D decomposition in the z-direction was employed (as indicated by the code when incorrect settings were used) in the form `1, 1, n_ranks`.

The problem size was varied with the `dims_global` parameter. The submitter has indicated that doubling the size in each direction increases the problem size by 8.

Core-level assessment

For a high-level analysis of a single core's performance, we look for the floating-point operation rate compared to the theoretical rate, $R_{theoretical}^{core}$ for a core. The hardware capabilities were determined with `likwid-bench` pinned to a single core on the first socket of a node:

```
likwid-bench -t peakflops -w S0:16kB:1
```

To analyse the performance of a single core, the code was run on one rank and thread with `likwid-perfctr` to get the observed floating point rate, with the [serial input indicated earlier](#):

```
likwid-perfctr -f -C 0 -g FLOPS_DP mpirun build-omp/xcompact serial.x3d
```

Despite running on a single core, the job was given access to an exclusive node with all cores and 64 GB memory available to it. In the absence of a dedicated serial configuration, a single MPI rank (and 1 thread per rank) configuration was used.

The observed and theoretical floating point rates were

Metric	MFLOPS
$R_{theoretical}^{core}$	8886.43
$R_{observed}^{core}$	3832.79

Based on these results, we determined that this software has a score of $C^{core} = \frac{R_{observed}}{R_{theoretical}} \approx 0.4312$. As per the rubric, the metric is classified as follows:

Peak FLOPS proportion	Description	Score
$C_{core} \geq 0.8$	Core performance does not require further analysis.	
$0.8 > C_{core} \geq 0.6$	Core performance may require further analysis as it may not be effectively using the hardware to its maximum potential.	
$0.6 > C_{core}$	Core performance requires in-depth analysis.	$C_{core} \approx 0.431$

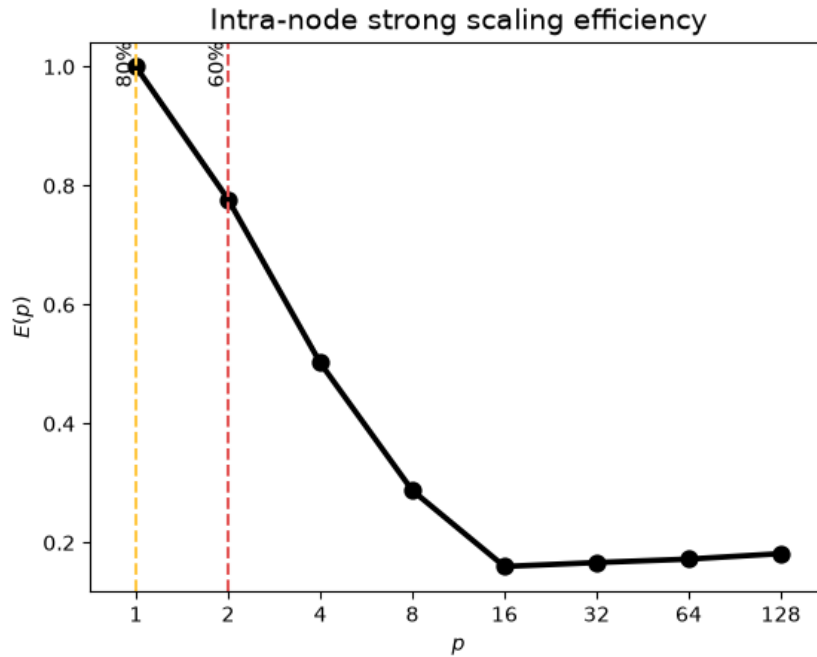
The metric shows that the code achieves less than 60% of a single core's achievable peak performance, indicating that this aspect requires deeper analysis.

Intra-node assessment

For a high-level analysis of the intra-node performance, we perform a strong scaling by fixing the problem size and increasing core allocation. For this code, we tested with core counts in powers of 2 from 1 to 128 over a complete node. The `dims_global` remained as indicated in the `base input`, while the `nproc_dir` was varied when changing the MPI ranks.

Cores	Ranks	Thread per rank	Time (s)	Parallel efficiency (%)
1	1	1	7100.00	1.00
2	1	2	4568.00	0.77
4	1	4	3534.70	0.50
8	1	8	3085.68	0.29
16	1	16	2776.57	0.16
32	2	16	1333.27	0.18
64	4	16	644.21	0.17

Cores	Ranks	Thread per rank	Time (s)	Parallel efficiency (%)
128	8	16	305.74	0.18



The parallel efficiency shows a steep drop from 1 to 16 cores, levelling around 16 to 18% from 32 to 128 cores. The 80% threshold is at 1 core and the 60% threshold is at 2 cores. As a proportion of the number of cores available on the node i.e. 128, the scores are:

$$C_{intra}^{80\%} = \frac{p_{critical,intra}^{80\%}}{p_{max,intra}} = \frac{1}{128} = 0.78125\%$$

and

$$C_{intra}^{60\%} = \frac{p_{critical,intra}^{60\%}}{p_{max,intra}} = \frac{2}{128} = 1.5625\%$$

Efficiency	Description	Score
$C_{intra}^{80\%} \geq 0.8$	Shared memory scaling does not require further analysis.	
$C_{intra}^{80\%} < 0.8 \wedge C_{intra}^{60\%} \geq 0.6$	Shared memory scaling may require analysis; scaling performance may be impacted when porting to newer chips with more cores.	
otherwise	Shared memory scaling requires in-depth analysis.	$C_{intra}^{80\%} \approx 1.78 \times 10^{-3}$ & $C_{intra}^{60\%} \approx 1.56 \times 10^{-2}$

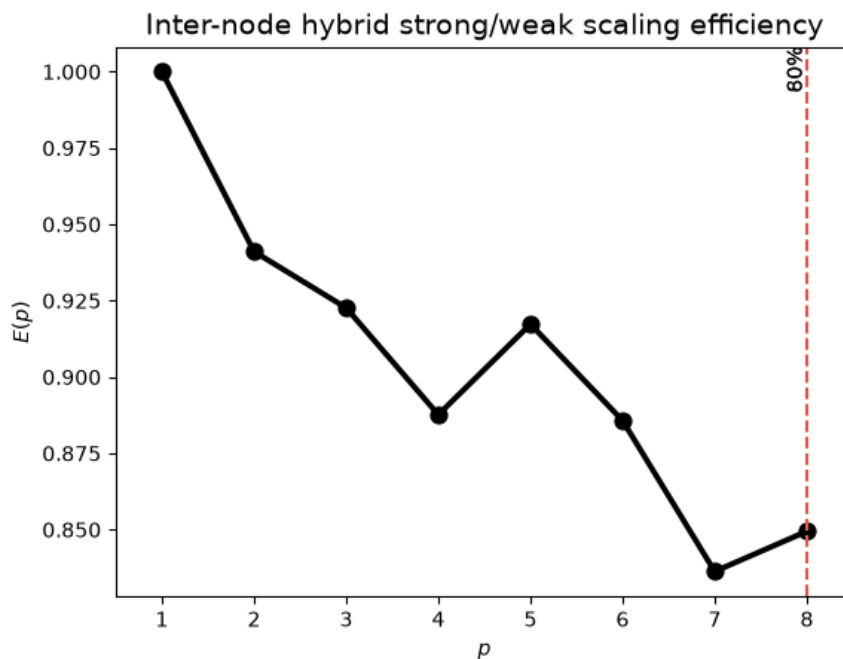
Based on the classification of the metrics and observations from the parallel efficiency plot, the code demonstrates unfavourable scaling within a node. The intra-node performance therefore requires further investigation.

Inter-node assessment

The inter-node performance is measured using a combination of strong and weak scaling analyses. For a given problem size, a strong scaling analysis is performed over increasing node counts, with the parallel efficiency computed with respect to the performance on a single node. When the efficiency drops below 60%, the problem size is increased relative to the initial size. The run is repeated with the new size for that node count, and the strong scaling analysis is continued over the remaining node counts. The step is repeated every time the efficiency falls below 60%.

For the initial problem size on one node, `dims_global` was set to 512, 512, 512. The runs were configured to have 8 MPI ranks per node (1 per NUMA domain) and 16 threads per rank. The total number of ranks used is `num_ranks_per_node x num_nodes`, which is used to set the `nproc_dir` parameter.

Nodes	Ranks	Threads per rank	Time (s)	Parallel efficiency (%)
1	8	16	2781.01	1
2	16	16	1477.58	0.94
3	24	16	1004.73	0.92
4	32	16	783.27	0.89
5	40	16	606.35	0.92
6	48	16	523.3	0.89
7	56	16	475.06	0.84
8	64	16	409.24	0.85



For 1 to 8 nodes on Hamilton, the parallel efficiency was never below 60% or 80%. As a result, the problem size did not require any increase for any node count. The metric, $C_{inter}^{80\%}$ was therefore 100%:

Efficiency	Description	Score
$C_{inter}^{80\%} \geq 0.8 \cdot C_{inter} \wedge C_{inter}^{80\%} + C_{inter}^{60\%} = C_{inter}$	Distributed memory scaling does not require further analysis.	$C_{inter}^{80\%} = 1.00$
$C_{intra}^{60\%} \geq 0.6 \cdot C_{inter}$	Distributed memory scaling may require further analysis.	
otherwise	Distributed memory scaling requires in-depth analysis.	

As the inter-node performance of the code demonstrated favourable scaling for the configuration analysis, this aspect does not require immediate attention.

GPU Assessment

We are currently working through the pre-assessment of the GPU build and so a GPU assessment is pending.

I/O Assessment

For the I/O performance, we look for the proportion of the runtime spent processing I/O requests. On Hamilton, the I/O performance was determined with the **darshan** tool. The code was run on 128 cores (1 node) with 8 ranks and 16 threads per rank. To obtain the I/O metrics from **darshan**, the following configurations were used:

```
module load gcc openmpi darshan
export DARSHAN_DIR=/apps/developers/tools/darshan/3.3.1/1/gcc-11.2-
openmpi-4.1.1/
export DARSHAN_LOGDIR=darshan_logs
LD_PRELOAD=${DARSHAN_DIR}/lib/libdarshan.so mpirun build-omp/xcompact one-
node.x3d
```

The following relevant I/O metrics were extracted from the **darshan** logs:

Metric	Time(s)
total_POSIX_F_READ_TIME	0.000664
total_POSIX_F_WRITE_TIME	0.001117

The I/O score, $C_{I/O}$, was computed as follows:

$$C_{I/O} = 1.0 - \frac{t_{read} + t_{write}}{t_{total}} = \frac{0.000664 + 0.001117}{7093.95} = 1 - 2.51058890561e - 07 \approx 0.999999748941 \approx 1.00$$

I/O proportion	Description	Score
$C_{I/O} \geq 0.8$	I/O does not require further analysis.	$C_{I/O} \approx 1.00$

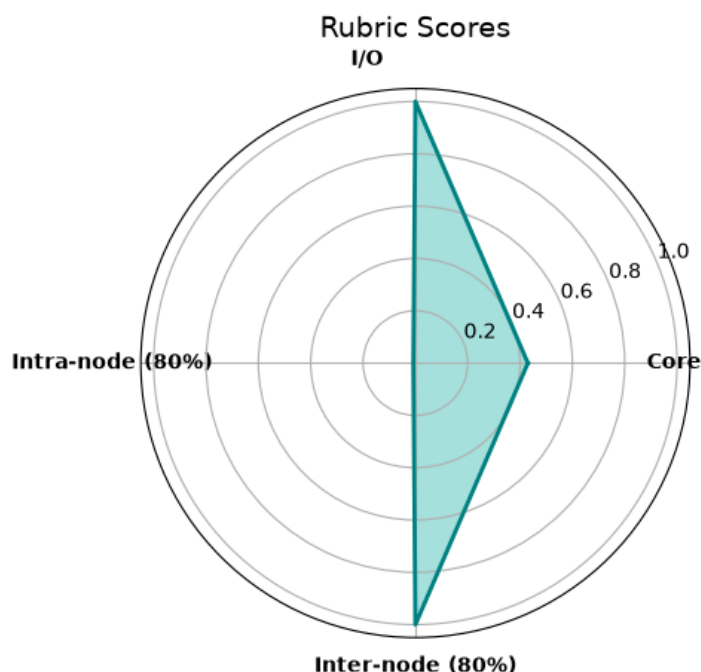
I/O proportion	Description	Score
$0.8 > C_{I/O} \geq 0.6$	I/O forms a significant part of the total runtime and may require further analysis.	
$0.6 > C_{I/O}$	I/O requires in-depth analysis.	

The metric indicates that for the given input the I/O has minimal impact on the runtime and so does not need to be addressed.

Summary

The following table collates the results of all above sections. These scores are indicative only, and cannot truly be compared to one another meaningfully without taking into account domain knowledge and methodological differences between them.

Result	Score	Metric result
Core	0.43	3832.79 MFLOPS
Intra-node (80%)	1.78×10^{-3}	1 core
Inter-node (80%)	1.00	7 nodes
GPU	-	-
I/O	1.00	2.51×10^{-7}



High-level assessment outcome

Based on the scores assigned, the [core-level performance](#) and the [intra-node performance](#) may be inhibiting the performance of the code. These aspects are therefore candidates for further investigation in a lower-level analysis. We note that varying the [nproc_dir](#) parameter, other inputs

such as `n_iters` or the assignment of MPI ranks and OpenMP threads may have a meaningful impact on the performance. While their assessment is out-of-scope for the high-level assessment, their analysis should be included in lower-level assessments.